claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_6c18d403-385\agent-aa78baa4b40767a56.jsonl`
- SHA-256: `1e53f92b07957e1fad3d0a61ec0480eabda3e802defe97d26e4c0a2b71dbc9db`
- Source modified: `2026-07-06T21:01:52+00:00`
- Imported at: `2026-07-08T16:00:06+00:00`
- project: `wf_6c18d403-385`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T20:56:00.637Z)

```
Reviewing GitHub PR #50 of the repo at C:/Users/avidu/Projects/Telegram ? "feat(policy): add F1
pipeline primitives" (branch feat/f1-policy-engine -> main).
It is the first F1 slice of ADR 0012 (policy pipeline, just merged as #40). +280/-14, 4 files:
- src/tg_video_filter/policy.py (NEW): PolicyContext, StageResult (+ pass_/fail helpers),
PolicyStage Protocol, evaluate_pipeline().
- src/tg_video_filter/db.py: NEW load_policy_config/save_policy_config (raw JSON dict over
policies.config_json);
  load_filter_config now uses model_validate(_policy_config_json_to_dict(...)) instead of
model_validate_json;
  save_filter_config now MERGES cfg.model_dump(mode='json') into the existing raw config dict
(preserving unknown
  keys) instead of replacing the whole blob with cfg.model_dump_json().
- tests/test_db.py + tests/test_policy.py: primitives tests + a preserve-unknown-keys regression
test.

A checked-out copy of the PR is at C:/Users/avidu/Projects/_wt-pr50 (read files there or via
'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f1-policy-engine:<path>').
Run repros with: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr50/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>  (build schema via
db._apply_schema on aiosqlite ":memory:").

BACKGROUND ? why this PR exists: ADR 0012 review found that
load_filter_config/save_filter_config would ERASE future
pipeline keys (topics/security_checks/deep_inspect) when a user runs /set, because save re-
serialized via FilterConfig
(which drops unknown keys). F1 fixes that persistence boundary. The main reviewer already
verified the happy path works
(pipeline keys survive a /set; flat fields update). Your job: find edge cases / bugs /
regressions the happy path missed.

Ground rules:
- READ actual code, cite file:line. Run repros. Authoritative gate ALREADY RUN: ruff clean, 501
passed @ 96.63%.
- Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario per finding. Rank
most-severe first.
- Severity: high (real bug/regression/data-loss), medium (edge case, latent, asymmetry), low
(typing/hygiene/scope-note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr50/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong
behaviour. REFUTED if
```

handled / can't occur / pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled.
Corrected severity.
FINDING (persistence): save_policy_config hardcodes name='default' but get_default_policy is
name-agnostic; a default policy named other than 'default' makes save raise IntegrityError and
become unsaveable
severity low | src/tg_video_filter/db.py:1333
summary: save_policy_config always upserts with name='default', is_default=True, but locates the
existing policy via get_default_policy (which matches is_default=1 regardless of name). If the
account's default policy has any other name, upsert_policy inserts a second is_default=1 row
(name conflict misses, default partial-unique index hits) and raises sqlite3.IntegrityError, so
the config can never be saved and pipeline keys are stranded.
scenario: An account's default policy is named e.g. 'my-custom-default' with is_default=1 (a
shape ADR 0011 permits ? the default is defined by is_default, not by name). A user runs /set:
save_filter_config -> save_policy_config -> upsert_policy(name='default', is_default=True). ON
CONFLICT(account_id,name) does not fire (names differ), so a fresh INSERT is attempted with
is_default=1, violating idx_policies_account_default. Verified repro (r4.py):
'save_filter_config -> IntegrityError: UNIQUE constraint failed: policies.account_id'. The
user's /set silently fails (raises) and the default policy's existing pipeline keys cannot be
updated.
evidence: db.py:1327 existing=get_default_policy (matches is_default=1 only, db.py:1285);
db.py:1333-1335 hardcodes name='default', is_default=True; upsert_policy conflict key is
(account_id,name) at db.py:1257. Repro r4.py reproduced the IntegrityError. Note: no code path
in this PR/repo creates a default policy with a non-'default' name (migration 003 and both shims
use 'default'), so this is currently latent/defensive only.
rec: Make save_policy_config target the existing default policy by its actual name
(existing.name if existing is not None else 'default') so it updates the row get_default_policy
found, instead of assuming the name is 'default'.

2. assistant (2026-07-06T20:56:02.967Z)

I'll verify this finding by reading the actual code and running a repro.

3. user (2026-07-06T20:56:03.877Z)

```
1270        await conn.commit()
1271        conn.row_factory = aiosqlite.Row
1272        async with conn.execute(
1273            "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1274            (policy.account_id, policy.name),
1275        ) as cur:
1276            row = await cur.fetchone()
1277        assert row is not None
1278        return _row_to_policy(row)
1279
1280
1281    async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy |
None:
1282        """Return the account's default policy (``is_default=1``)."""
1283        conn.row_factory = aiosqlite.Row
1284        async with conn.execute(
1285            "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1286            (account_id,),
1287        ) as cur:
1288            row = await cur.fetchone()
1289        return _row_to_policy(row) if row else None
1290
1291
1292    def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1293        """Decode a policy config JSON object.
1294
1295        ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1296        expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
1297        nested stage config can coexist.
1298        """
1299        decoded = json.loads(raw)
```

```
1300        if not isinstance(decoded, dict):
1301            raise ValueError("policy config_json must decode to a JSON object")
1302        return decoded
1303
1304
1305    def _policy_config_dict_to_json(config: dict[str, object]) -> str:
1306        """Encode policy config deterministically and compactly."""
1307        return json.dumps(config, separators=(",", ":"), sort_keys=True)
1308
1309
1310    async def load_policy_config(conn: aiosqlite.Connection, account_id: int) -> dict[str,
object]:
1311        """Load the raw default policy config as a JSON object.
1312
1313        Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
1314        as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
1315        through the flat ``FilterConfig`` model.
1316        """
1317        policy = await get_default_policy(conn, account_id)
1318        if policy is None:
1319            return FilterConfig().model_dump(mode="json")
1320        return _policy_config_json_to_dict(policy.config_json)
1321
1322
1323    async def save_policy_config(
1324        conn: aiosqlite.Connection, account_id: int, config: dict[str, object]
1325    ) -> None:
1326        """Persist the raw default policy config object."""
1327        existing = await get_default_policy(conn, account_id)
1328        await upsert_policy(
1329            conn,
1330            Policy(
1331                id=existing.id if existing is not None else None,
1332                account_id=account_id,
1333                name="default",
1334                config_json=_policy_config_dict_to_json(config),
1335                is_default=True,
1336                created_at=(existing.created_at if existing is not None else _now()),
1337                updated_at=_now(),
1338            ),
1339        )
1340
1341
1342    # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1343
1344
1345    async def upsert_destination(conn: aiosqlite.Connection, dest: Destination) ->
Destination:
1346        """Insert or update a destination row by (account, provider, target)."""
1347        await conn.execute(
1348            """INSERT INTO destinations
1349                 (account_id, provider, target, kind, title, is_output, created_at)
1350               VALUES (?, ?, ?, ?, ?, ?, ?)
1351               ON CONFLICT(account_id, provider, target) DO UPDATE SET
1352                 kind=excluded.kind,
1353                 title=excluded.title,
1354                 is_output=excluded.is_output""",
1355            (
1356                dest.account_id,
1357                dest.provider,
1358                dest.target,
1359                dest.kind,
```

4. assistant (2026-07-06T20:56:06.309Z)

I need to check the schema for the partial-unique index and verify `upsert_policy`'s conflict handling, then run the repro.

5. user (2026-07-06T20:56:07.302Z)

```
1240
1241
1242    # --- Policy repos (ADR 0011, Option 2A) -----------------------------------
1243
1244
1245    async def upsert_policy(conn: aiosqlite.Connection, policy: Policy) -> Policy:
1246        """Insert or update a policy row by ``(account_id, name)``.
1247
1248        Exactly one policy per account should carry ``is_default=True``; callers
1249        are responsible for flipping the previous default before flipping a new
1250        one (enforced by a partial unique index landed by future work, not in the
1251        schema ? SQLite ``CHECK`` can't express "at most one per group").
1252        """
1253        await conn.execute(
1254            """INSERT INTO policies (account_id, name, config_json, is_default,
1255                                     created_at, updated_at)
1256               VALUES (?, ?, ?, ?, ?, ?)
1257               ON CONFLICT(account_id, name) DO UPDATE SET
1258                 config_json=excluded.config_json,
1259                 is_default=excluded.is_default,
1260                 updated_at=excluded.updated_at""",
1261            (
1262                policy.account_id,
1263                policy.name,
1264                policy.config_json,
1265                int(policy.is_default),
1266                policy.created_at.isoformat(),
1267                policy.updated_at.isoformat(),
1268            ),
1269        )
1270        await conn.commit()
1271        conn.row_factory = aiosqlite.Row
1272        async with conn.execute(
1273            "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1274            (policy.account_id, policy.name),
1275        ) as cur:
1276            row = await cur.fetchone()
1277        assert row is not None
1278        return _row_to_policy(row)
1279
```

6. assistant (2026-07-06T20:56:10.564Z)

Note the docstring says "partial unique index landed by future work, not in the schema". Let me check whether the partial-unique index actually exists in the schema. This is critical ? if there's no index, the IntegrityError claimed in the finding cannot occur.

7. user (2026-07-06T20:56:11.487Z)

```
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:134:    is_default  INTEGER NOT NULL
DEFAULT 0,
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:141:CREATE UNIQUE INDEX IF NOT EXISTS
idx_policies_account_default
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:142:ON policies(account_id) WHERE
is_default = 1;
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:619:    #    (Option 2A).
is_default=1 so the legacy load_filter_config shim reads
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:623:        INSERT OR IGNORE INTO
policies (account_id, name, config_json, is_default,
```

```
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:636:        "CREATE UNIQUE INDEX IF
NOT EXISTS idx_policies_account_default "
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:637:        "ON policies(account_id)
WHERE is_default = 1"
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:723:        JOIN policies p ON
p.account_id = s.account_id AND p.is_default = 1
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:887:
is_default=bool(row["is_default"]),
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1248:    Exactly one policy per
account should carry ``is_default=True``; callers
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1254:        """INSERT INTO policies
(account_id, name, config_json, is_default,
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1259:
is_default=excluded.is_default,
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1265:
int(policy.is_default),
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1282:    """Return the account's
default policy (``is_default=1``)."""
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1285:        "SELECT * FROM policies
WHERE account_id = ? AND is_default = 1",
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1335:            is_default=True,
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1753:
``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\models.py:192:    Exactly one policy per
account carries ``is_default=True``; the legacy
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\models.py:204:    is_default: bool = Field(
```

8. assistant (2026-07-06T20:56:16.401Z)

The partial-unique index DOES exist (db.py:141-142, and applied at 636-637). So the IntegrityError mechanism is real. Now let me verify the scenario is in-diff. The key question: is `save_policy_config` / this whole persistence path part of THIS PR's diff, and does the described scenario actually reproduce? Let me run the repro.

9. assistant (2026-07-06T20:56:23.183Z)

```
import asyncio, aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Policy
from datetime import datetime, timezone

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    # create an account row prerequisite? check accounts table
    # Insert a default policy named 'my-custom-default' with is_default=1
    now = datetime.now(timezone.utc)
    await conn.execute(
        "INSERT INTO policies (account_id, name, config_json, is_default, created_at,
updated_at) "
        "VALUES (?,?,?,?,?,?)",
        (42, "my-custom-default", '{"foo": 1}', 1, now.isoformat(), now.isoformat()),
    )
    await conn.commit()
    print("seeded custom-named default")
    try:
        await db.save_policy_config(conn, 42, {"foo": 2, "topics": ["x"]})
        print("save_policy_config OK")
        cfg = await db.load_policy_config(conn, 42)
        print("loaded:", cfg)
        # count rows
        async with conn.execute("SELECT name,is_default FROM policies WHERE account_id=42") as
cur:
            print("rows:", await cur.fetchall())
    except Exception as e:
```

```
        print(f"save_policy_config -> {type(e).__name__}: {e}")

asyncio.run(main())
```

10. user (2026-07-06T20:56:23.184Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

11. user (2026-07-06T20:56:27.589Z)

```
(Bash completed with no output)
```

12. assistant (2026-07-06T20:56:33.889Z)

```
import asyncio, aiosqlite
from tg_video_filter import db
from datetime import datetime, timezone

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    now = datetime.now(timezone.utc)
    await conn.execute(
        "INSERT INTO policies (account_id, name, config_json, is_default, created_at,
updated_at) "
        "VALUES (?,?,?,?,?,?)",
        (42, "my-custom-default", '{"foo": 1}', 1, now.isoformat(), now.isoformat()),
    )
    await conn.commit()
    print("seeded custom-named default (is_default=1)")
    try:
        await db.save_policy_config(conn, 42, {"foo": 2, "topics": ["x"]})
        print("save_policy_config OK")
        cfg = await db.load_policy_config(conn, 42)
        print("loaded:", cfg)
        async with conn.execute("SELECT name,is_default FROM policies WHERE account_id=42") as
cur:
            print("rows:", await cur.fetchall())
    except Exception as e:
        print(f"save_policy_config -> {type(e).__name__}: {e}")

asyncio.run(main())
```

13. user (2026-07-06T20:56:33.920Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/r4.py (file state is
current in your context ? no need to Read it back)
```

14. user (2026-07-06T20:58:40.909Z)

```
Command running in background with ID: brx0v806b. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\brx0v806b.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

15. user (2026-07-06T20:58:48.237Z)

```
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r4.py", line 26, in <module>
    asyncio.run(main())
    ~~~~~~~~~~~^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
```

```
195, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
           ~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r4.py", line 9, in main
    await conn.execute(
    ...<3 lines>...
    )
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223,
in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160,
in _execute
    return await future
           ^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in
_connection_worker_thread
    result = function()
sqlite3.IntegrityError: FOREIGN KEY constraint failed
```

16. assistant (2026-07-06T20:58:52.468Z)

There's a FK constraint requiring an accounts row. Let me check how existing tests seed an account.

17. user (2026-07-06T20:58:54.027Z)

```
39:async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) -> User:
75:    created = await _seed_user(conn)
89:    await _seed_user(conn)
109:async def _seed_account(conn: aiosqlite.Connection, aid: int = USER_ID) -> Account:
113:async def test_upsert_and_get_account(conn: aiosqlite.Connection) -> None:
126:async def test_get_account_returns_none_when_absent(conn: aiosqlite.Connection) -> None:
130:async def test_get_account_by_telegram_id_resolves_bot_from_id(
143:    await _seed_account(conn)
183:async def test_bot_only_account_has_no_session_and_is_not_a_user(
205:    await _seed_user(conn)
228:    await _seed_user(conn)
255:    await _seed_user(conn)
272:    await _seed_user(conn)
287:    await _seed_user(conn)
333:    await _seed_user(conn)
350:    await _seed_user(conn)
356:    await _seed_user(conn)
381:    await _seed_user(conn)
405:    await _seed_user(conn)
431:    await _seed_user(conn)

[Showing results with pagination = limit: 20]
```

18. user (2026-07-06T20:58:57.042Z)

```
30      @pytest.fixture
31      async def conn() -> aiosqlite.Connection:
32          """An in-memory SQLite connection with the schema applied."""
```

```
33          c = await aiosqlite.connect(":memory:")
34          await db._apply_schema(c)
35          yield c
36          await c.close()
37
38
39      async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) -> User:
40          return await db.upsert_user(conn, User(id=uid, phone="+15555550100"))
41
42
43      # --- schema / init ----------------------------------------------------------
44
45
46      async def test_init_db_creates_tables(tmp_path: object) -> None:
47          db_path = str(tmp_path) + "/test.db"  # type: ignore[operator]
48          await db.init_db(db_path)
49          # Re-open and confirm all expected tables exist. Post-ADR-0011 the
50          # provider-neutral tables are sources/content_items/policies (plus the
51          # empty Track-C tables destinations/routing_rules/delivery_attempts);
52          # the legacy channels/videos/filter_config are absent on fresh DBs
53          # (they only exist as *_deprecated on migrated legacy DBs).
54          async with (
55              aiosqlite.connect(db_path) as c,
56              c.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name") as
cur,
57          ):
58              names = [r[0] for r in await cur.fetchall()]
59          assert set(names) >= {
60              "accounts",
61              "telegram_sessions",
62              "sources",
63              "content_items",
64              "policies",
65              "destinations",
66              "routing_rules",
67              "delivery_attempts",
68          }
69
70
71      # --- user repos -------------------------------------------------------------
72
73
74      async def test_upsert_and_get_user(conn: aiosqlite.Connection) -> None:
75          created = await _seed_user(conn)
76          assert created.id == USER_ID
77          assert created.enabled is True
78
79          fetched = await db.get_user(conn, USER_ID)
80          assert fetched is not None
81          assert fetched.phone == "+15555550100"
82
83
84      async def test_get_user_returns_none_when_absent(conn: aiosqlite.Connection) -> None:
85          assert await db.get_user(conn, 999999) is None
86
87
88      async def test_upsert_user_updates_existing(conn: aiosqlite.Connection) -> None:
89          await _seed_user(conn)
90          # Disable the user via an upsert.
91          await db.upsert_user(conn, User(id=USER_ID, phone="+15555550200", enabled=False))
92          fetched = await db.get_user(conn, USER_ID)
93          assert fetched is not None
94          assert fetched.enabled is False
95          assert fetched.phone == "+15555550200"
96
```

```
97
98      async def test_list_enabled_users_filters_disabled(conn: aiosqlite.Connection) -> None:
99          await db.upsert_user(conn, User(id=1, enabled=True))
100         await db.upsert_user(conn, User(id=2, enabled=False))
101         await db.upsert_user(conn, User(id=3, enabled=True))
102         enabled = await db.list_enabled_users(conn)
103         assert sorted(u.id for u in enabled) == [1, 3]
104
105
106     # --- account + session repos (ADR 0010) ----------------------------------
107
108
109     async def _seed_account(conn: aiosqlite.Connection, aid: int = USER_ID) -> Account:
```

19. assistant (2026-07-06T20:58:59.629Z)

Let me update the repro to seed a user/account first.

20. user (2026-07-06T20:59:00.771Z)

```
109     async def _seed_account(conn: aiosqlite.Connection, aid: int = USER_ID) -> Account:
110         return await db.upsert_account(conn, Account(id=aid, telegram_id=aid))
111
112
113     async def test_upsert_and_get_account(conn: aiosqlite.Connection) -> None:
114         created = await db.upsert_account(
115             conn, Account(id=USER_ID, telegram_id=USER_ID, display_name="Alice")
116         )
117         assert created.id == USER_ID
118         assert created.telegram_id == USER_ID
119         assert created.provider == "telegram"
120
121         fetched = await db.get_account(conn, USER_ID)
122         assert fetched is not None
123         assert fetched.display_name == "Alice"
```

21. user (2026-07-06T20:59:07.234Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/r4.py has been updated successfully. (file state is current in your context ? no need to Read it back)

22. user (2026-07-06T21:01:13.556Z)

Command running in background with ID: b4svo1ksg. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b4svo1ksg.output. You will be notified when it completes. To check interim output, use Read on that file path.

23. user (2026-07-06T21:01:22.846Z)

```
seeded custom-named default (is_default=1)
save_policy_config -> IntegrityError: UNIQUE constraint failed: policies.account_id
```

24. assistant (2026-07-06T21:01:28.242Z)

The IntegrityError reproduces exactly as described. The mechanism is fully confirmed: with a default policy named other than 'default', `save_policy_config` hardcodes `name="default"`, the `ON CONFLICT(account_id,name)` misses (names differ), a fresh INSERT with `is_default=1` is attempted, and the partial-unique index `idx_policies_account_default` (db.py:141-142) fires.

Now the critical severity question: can this scenario actually arise? The finding itself concedes "no code path in this PR/repo creates a default policy with a non-'default' name" ? let me verify that claim, since it's the whole basis for low severity.

25. user (2026-07-06T21:01:29.528Z)

```
618:    # 3. Backfill policies from filter_config: one 'default' policy per account
625:        SELECT user_id, 'default', config_json, 1,
1333:            name="default",
```

26. user (2026-07-06T21:01:35.550Z)

```
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\models.py:186:class Policy(BaseModel):
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\models.py:192:    Exactly one policy per
account carries ``is_default=True``; the legacy
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:623:        INSERT OR IGNORE INTO
policies (account_id, name, config_json, is_default,
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:882:    return Policy(
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1245:async def upsert_policy(conn:
aiosqlite.Connection, policy: Policy) -> Policy:
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1248:    Exactly one policy per
account should carry ``is_default=True``; callers
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1254:        """INSERT INTO policies
(account_id, name, config_json, is_default,
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1328:    await upsert_policy(
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1330:        Policy(
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1335:            is_default=True,
C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py:1843:    "upsert_policy",
```

27. assistant (2026-07-06T21:01:43.112Z)

The only two paths that create a default policy in the entire src tree:
1. Migration backfill (db.py:625): hardcodes `'default'`.
2. `save_policy_config` itself (db.py:1333): hardcodes `"default"`.

There is no code path anywhere in the repo that creates a default policy with a non-'default' name. The `is_default=True` write via `upsert_policy` happens only from `save_policy_config` (name='default') and the migration (name='default'). So a non-'default' default policy can only exist via external/manual DB manipulation, which is not something the application produces.

This confirms the finding is accurate but purely latent/defensive ? the wrong behavior genuinely ships in this PR's diff (the hardcoded `name="default"` at db.py:1333 was introduced by this PR), and it does produce the described IntegrityError. However, the triggering precondition (a default policy with a non-default name) cannot be produced by any code path, so it cannot occur in practice. The finding itself correctly discloses this.

The recommendation (`existing.name if existing is not None else "default"`) is a sound, cheap robustness improvement. Severity `low` is correct ? it's a defensive/latent asymmetry with no reachable trigger.

28. user (2026-07-06T21:01:52.053Z)

Structured output provided successfully